

FPGA Based Vending Machine

23ECE383 VLSI Design Laboratory

B. Tech. ECE

Batch: **C2**

2025-2026 Odd Semester

CB.EN.U4ECE23246	SRICHARAN R P
CB.EN.U4ECE23249	SUJITH KRISHNA P G
CB.EN.U4ECE23260	VIPIN RITHIK R L

Faculty In-charges: Dr. E. Prabhu, Dr. B. Yamuna

Signature of the Faculty:

Department of Electronics and Communication Engineering

Amrita School of Engineering

Amrita Vishwa Vidyapeetham

Amritanagar, Coimbatore – 641112

1. Objective:

The objective of this project is to design a vending machine using the Basys3 FPGA board and a keypad that:

- Accepts \$1, \$2, and \$5 coins.
- Dispenses up to 5 different products.
- Returns change if excess money is inserted.
- Refunds money if the cancel option is chosen.
- Notifies the customer if stock is unavailable.
- Uses a **7-segment display** to show product price, remaining money, and refund/change.

2. Software hardware used:

Hardware:

- Basys3 FPGA board
- PMOD Keypad (for input)

Software:

- Xilinx Vivado
- ModelSim - Intel FPGA Starter Edition 10.5b (Quartus Prime 18.1)

3. Overview of the system:

The vending machine system is built around the Finite State Machine (FSM) model, which ensures a systematic and predictable behaviour of the machine. The machine accepts inputs from a PMOD Keypad attached to the Basys3 FPGA board, where users can select from five available products. Each product has a predefined price, and the machine only accepts \$1, \$2, and \$5 denominations.

In the proposed vending machine design, each product is assigned a fixed price and an initial stock quantity. The system parameters are defined as follows:

- **Product A:** Price = \$6, Quantity = 10
- **Product B:** Price = \$5, Quantity = 4
- **Product C:** Price = \$9, Quantity = 1
- **Product D:** Price = \$4, Quantity = 3
- **Product E:** Price = \$11, Quantity = 0 (out of stock)

The FSM governs the following sequence of operations:

1. **Reset State:** Initializes the system and prepares it for a new transaction.
2. **Product Selection State:** The customer selects a product using the keypad. The machine checks availability. If the selected product is out of stock, the system immediately goes to the “No Stock” state.
3. **Money Amount State:** The machine waits for the user to insert money. The inserted amount is displayed on the 7-segment display, along with the balance still required.
4. **Dispense Item State:** If the inserted amount matches or exceeds the product price, the product is dispensed.
5. **Change State:** If excess money has been inserted, the machine calculates and dispenses the correct change.
6. **Refund State:** If the cancel button is pressed at any point before completion, the inserted money is refunded.
7. **Stock State (No Stock):** If the selected item is unavailable, the system informs the user and returns to reset.

The outputs of the system are displayed on the 7-segment display of the Basys3 FPGA board, which shows the amount inserted, remaining balance, and refund/change. The system also signals when an item is dispensed.

4. Block diagram:

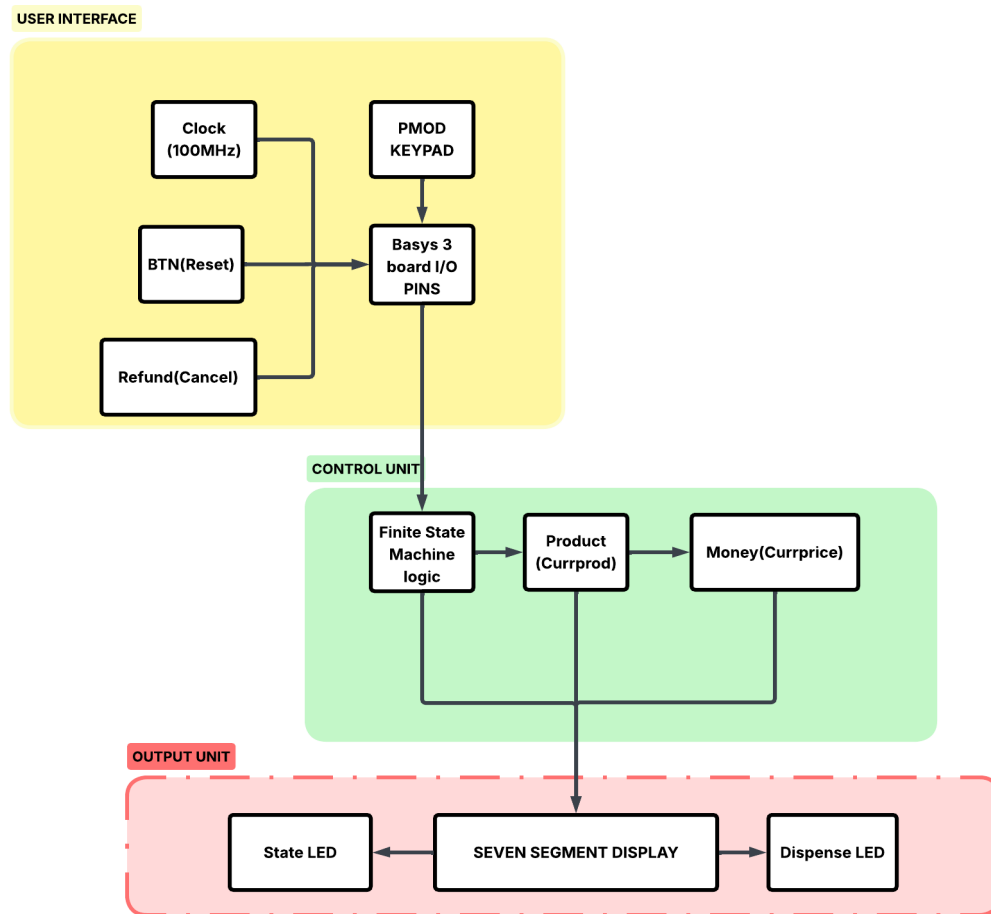


Fig. 1 Overview of the System

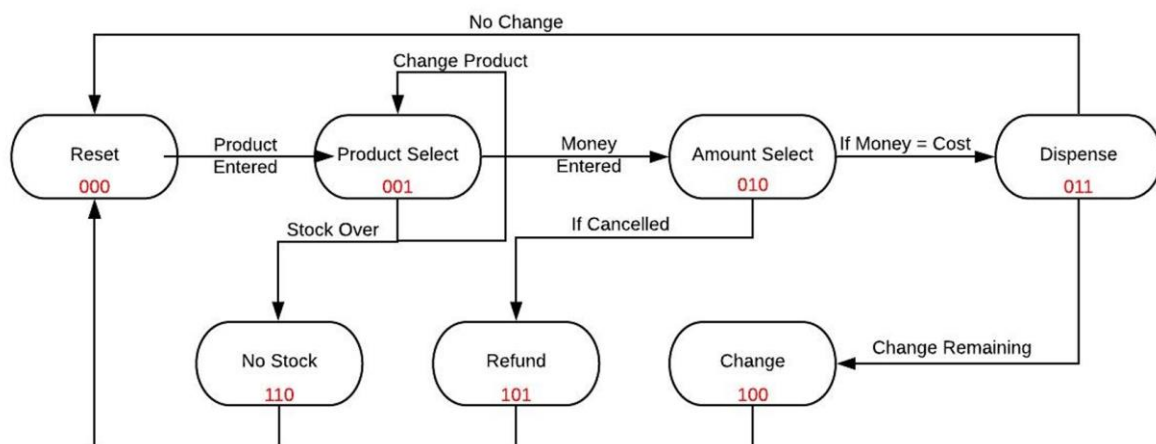


Fig. 2 State Diagram

5. Introduction:

Vending machines are an integral part of modern daily life, providing quick and convenient access to products without requiring human assistance. Despite their popularity, many existing vending machines have significant limitations. A common complaint among users is that machines often require the exact amount of money and do not return change. Another issue is that once money has been inserted, the customer often cannot cancel the transaction to get a refund. Additionally, some machines do not notify customers when a product is out of stock, leading to user dissatisfaction.

To address these challenges, our project focuses on designing a feature-rich vending machine using an FPGA-based system. The design integrates both hardware and software modules to create a vending machine that not only dispenses products but also:

- Returns change if extra money is inserted.
- Allows cancellation and refunds the amount already inserted.
- Notifies the user when a selected product is out of stock.
- Clearly displays transaction details using the onboard 7-segment display.

By overcoming the shortcomings of traditional vending machines, our design demonstrates how digital systems and state machine modelling can be applied to build efficient, customer-friendly, and reliable machines.

6. Complete Code with comments:

TOP.V:

```
// Products = 5, Amount Selections = 3
```

```
// No. of FSM States = 7
```

```
module top(clk, row, col, an, seg, led_reset, led_check, ps, dispense, money, btn, refnd, dp);
```

```
    input wire clk;
```

```
    input wire [3:0] row;
```

```
    input wire btn;
```

```
    input wire refnd;
```

```

output [3:0] an;
output reg dp = 1'b1;
output [6:0] seg;
output led_reset;
output wire [3:0] col;
output wire dispense;
output wire [2:0] ps;
output wire led_check;
output wire [3:0] money;


wire clk_1ms, clk_1s;
wire [3:0] LED;
wire [3:0] change;
wire [2:0] ns;
wire [6:0] dout;
wire [3:0] CurrProd;
wire [3:0] CurrPrice;


// Clock divider generates 1ms and 1s clocks from main clk
clock_divider c1 (.clk(clk), .clk_1ms(clk_1ms), .clk_1s(clk_1s));


// Keypad interface module – captures product/money input
Keypad k1(.clk_1ms(clk_1ms), .row(row), .col(col), .LED(LED), .clk_1s(clk_1s));


// FSM module – main vending machine logic (Next State Logic)
NSL n1(.clk_1ms(clk_1s), .LED(LED), .dispense(dispense),
.change(change), .led_reset(led_reset), .ps(ps), .CurrProd(CurrProd), .CurrPrice(CurrPrice),
money(money), .btn(btn), .refnd(refnd));


// Seven-segment display module – shows product, price, or money
Sevenseg s1(.clk_1ms(clk_1ms), .ps(ps),
.anode(an), .dout(seg), .LED(LED), .led_check(led_check), .CurrProd(CurrProd), .CurrPrice
(CurrPrice), .money(money));
endmodule

```

CLOCK_DIVIDER.V:

```
module clock_divider(  
    clk,  
    clk_1ms, clk_1s  
);  
    reg [27:0] i = 0;  
    reg [27:0] j = 0;  
    input wire clk;  
    output reg clk_1ms = 0;  
    output reg clk_1s = 0;  
  
    // Generate 1ms clock  
    always @(posedge clk)  
    begin  
        if (i == 49999) // Divide factor for 1ms period (assuming 100MHz input clock)  
        begin  
            i <= 0;  
            clk_1ms = ~clk_1ms; // Toggle 1ms clock  
        end  
        else i <= i + 1;  
    end  
  
    // Generate 1s clock  
    always @(posedge clk)  
    begin  
        if (j == 2499999) // Divide factor for 1s period  
        begin  
            j <= 0;  
            clk_1s = ~clk_1s; // Toggle 1s clock  
        end  
        else j <= j + 1;  
    end  
endmodule
```

KEYPAD.V:

```
module Keypad(clk_1ms, row, col, LED, clk_1s);
```

```
    input wire clk_1ms;
```

```
    input wire clk_1s;
```

```
    input wire [3:0] row;
```

```
    output reg [3:0] col = 4'b0000;
```

```
    output reg [3:0] LED;
```

```
    always @ (posedge clk_1s)
```

```
    begin
```

```
        case(col)
```

```
            4'b0111:
```

```
            begin
```

```
                if(row == 4'b0111)//D
```

```
                LED = 13;
```

```
                else if(row == 4'b1011)//C
```

```
                LED = 12;
```

```
                else if(row == 4'b1101)//B
```

```
                LED = 11;
```

```
                else if(row == 4'b1110)//A
```

```
                LED = 10;
```

```
                else LED = 0;
```

```
            end
```

```
            4'b1011:
```

```
            begin
```

```
                if(row == 4'b0111)//e
```

```
                LED = 14;
```

```
                else if(row == 4'b1011)//9
```

```
                LED = 9;
```

```
                else if(row == 4'b1101)//6
```

```
                LED = 6;
```

```
                else if(row == 4'b1110)//3
```

```
                LED = 3;
```

```

        else LED = 0;
    end
4'b1101:
begin
    if(row == 4'b0111)//F
        LED = 15;
    else if(row == 4'b1011)//8
        LED = 8;
    else if(row == 4'b1101)//5
        LED = 5;
    else if(row == 4'b1110)//2
        LED = 2;
    else LED = 0;
end
4'b1110:
begin
    if(row == 4'b0111)//0
        LED = 0;
    else if(row == 4'b1011)//7
        LED = 7;
    else if(row == 4'b1101)//4
        LED = 4;
    else if(row == 4'b1110)//1
        LED = 1;
    else LED = 0;
end
    default: LED = 0;//changed from 0 to Z// works only for col not rows
endcase
end

always @(posedge clk_1s)
begin
    case (col)
4'b0111: col <= 4'b1011;

```

```

        4'b1011: col <= 4'b1101;
        4'b1101: col <= 4'b1110;
        4'b1110: col <= 4'b0111;
        default: col <= 4'b0111;
    endcase
end
endmodule

```

NSL.V:

```

module NSL(
clk_1ms, LED, dispense, change, led_reset, ps, CurrProd, CurrPrice, money, btn, clk_1s,
refnd);
    input wire clk_1ms;
    input wire clk_1s;
    input wire [3:0] LED;
    output reg dispense;
    output reg [3:0] change;
    output reg led_reset;
    input wire btn;
    input wire refnd;

    reg flag = 1'b0;
    reg flag2 = 1'b0;

    output reg [2:0] ps;
    reg [2:0] ns;

    // State encoding
    parameter Reset = 0;
    parameter ProdSel = 1;
    parameter AmtSel = 2;
    parameter Disp = 3;
    parameter Chng = 4;
    parameter Refund = 5;

```

```

parameter Stock = 6;

// Product prices
parameter A_price = 06;
parameter B_price = 05;
parameter C_price = 09;
parameter D_price = 04;
parameter E_price = 11;

// Product quantities
reg [3:0] A_quantity = 10;
reg [3:0] B_quantity = 4;
reg [3:0] C_quantity = 1;
reg [3:0] D_quantity = 3;
reg [3:0] E_quantity = 0;

output reg [3:0] CurrProd;
output reg [3:0] CurrPrice;
output reg [3:0] money; //Max value it can hold is 15$

//=====
// State update logic
//=====
always @ (posedge clk_1ms)
ps <= ns; // Next state becomes present state every clock cycle

//=====
// Main FSM
//=====
always @ (posedge clk_1ms)
begin
case(ps)
Reset:
begin

```

```

led_reset = 1'b1; // Reset all LEDs
if (LED == 4'hA || LED == 4'hB || LED == 4'hC || LED == 4'hD || LED == 4'hE)
begin
    ns <= ProdSel; // Move to product selection state
    CurrProd <= LED; // Store selected product
end
else
begin
    flag = 1'b0;
    flag2 = 1'b0;
    money = 0;
    change = 0;
    dispense = 1'b0;
    CurrPrice = 4'bzzzz;
end
end

ProdSel:
begin
    if (LED == 4'hA || LED == 4'hB || LED == 4'hC || LED == 4'hD || LED == 4'hE)
begin
    CurrProd = LED; // Update current product
    ns <= ProdSel;
end
else if (LED == 4'h1 || LED == 4'h2 || LED == 4'h5) // Input money
begin
    ns <= AmtSel;
    money <= money + LED; // Add money
end

// Assign price based on product
case (CurrProd)
    4'hA: CurrPrice = A_price;
    4'hB: CurrPrice = B_price;

```

```

    4'hC: CurrPrice = C_price;
    4'hD: CurrPrice = D_price;
    4'hE: CurrPrice = E_price;
    default: CurrPrice = 0;
endcase

// Check for out-of-stock condition
case (CurrProd)
    4'hA: if (A_quantity == 4'h0) ns <= Stock;
    4'hB: if (B_quantity == 4'h0) ns <= Stock;
    4'hC: if (C_quantity == 4'h0) ns <= Stock;
    4'hD: if (D_quantity == 4'h0) ns <= Stock;
    4'hE: if (E_quantity == 4'h0) ns <= Stock;
    default : ns <= ProdSel;
endcase
end

AmtSel:
begin
    led_reset = ~led_reset; // Blink LED while waiting for money
    if (refnd)
        ns <= Refund; // Go to refund state if refund requested

    // Money input handling
    case (LED)
        4'h1: money <= money + 1;
        4'h2: money <= money + 2;
        4'h5: money <= money + 5;
        default: money <= money;
    endcase

    if (money >= CurrPrice)
        ns <= Disp; // Enough money, move to dispense
    end
end

```

```

Disp:
begin
    dispense <= 1'b1; // Dispense product LED ON

    // Reduce quantity only once
    if (flag2 == 1'b0)
    begin
        case(CurrProd)
            4'hA: A_quantity = A_quantity - 1;
            4'hB: B_quantity = B_quantity - 1;
            4'hC: C_quantity = C_quantity - 1;
            4'hD: D_quantity = D_quantity - 1;
            4'hE: E_quantity = E_quantity - 1;
        endcase
    end
    flag2 = 1'b1;

    if (btn)
        ns <= Reset; // Reset when button pressed
    else if (money > CurrPrice)
        ns <= Chng; // Go to change state if extra money
    else if (money == CurrPrice)
        ns <= Disp; // Stay here till button pressed
    end

Chng:
begin
    if (flag == 1'b0)
        change <= (money - CurrPrice); // Return extra money once

    if (btn)
        ns <= Reset;
    else

```

```

        begin
            ns <= Chng;
            flag = 1'b1;
        end
    end

    Refund:
    begin
        change <= money; // Refund entire amount
        if (btn)
            ns <= Reset;
        else ns <= Refund;
    end

    Stock:
    begin
        if (btn)
            ns <= Reset; // Reset if stock empty and button pressed
        else ns <= Stock;
    end

    default:
    begin
        ns <= Reset;
        led_reset = 1'b0;
    end
endcase
end
endmodule

```

SEVENSEG.V:

```

module sevenseg(clk_1ms, ps, anode, dout, LED, led_check, CurrPrice, CurrProd,
money);
    input wire clk_1ms;

```

```

input wire [2:0] ps;
input wire [3:0] LED;
input wire [3:0] CurrProd;
input wire [3:0] CurrPrice;
input wire [3:0] money;
output reg [3:0] anode;
output reg [6:0] dout;
output reg led_check;

parameter Reset = 0;
parameter ProdSel = 1;
parameter AmtSel = 2;
parameter Disp = 3;
parameter Chng = 4;
parameter Refund = 5;
parameter Stock = 6;

function [6:0] prod_disp;
input [3:0] CurrProd;
input [3:0] CurrPrice;
input [3:0] anode;

if (anode == 4'b1110)
begin
case (CurrProd)
4'hA : prod_disp = 7'b0001000; // A
4'hB : prod_disp = 7'b0000011; // B
4'hC : prod_disp = 7'b1000110; // C
4'hD : prod_disp = 7'b0100001; // D
4'hE : prod_disp = 7'b0000110; // E
default: prod_disp = 7'b1111111;
endcase
end
else if (anode == 4'b1101)

```

```

case(CurrPrice)
    4'h0 : prod_disp = 7'b1000000; // 0
    4'h1 : prod_disp = 7'b1111001; // 1
    4'h2 : prod_disp = 7'b0100100; // 2
    4'h3 : prod_disp = 7'b0110000; // 3
    4'h4 : prod_disp = 7'b0011001; // 4
    4'h5 : prod_disp = 7'b0010010; // 5
    4'h6 : prod_disp = 7'b0000010; // 6
    4'h7 : prod_disp = 7'b1111000; // 7
    4'h8 : prod_disp = 7'b0000000; // 8
    4'h9 : prod_disp = 7'b0010000; // 9
    4'hA : prod_disp = 7'b1000000; // Custom values of lsb for A
    4'hB : prod_disp = 7'b1111001; //          B
    4'hC : prod_disp = 7'b0100100; //          C
    4'hD : prod_disp = 7'b0110000; //          D
    4'hE : prod_disp = 7'b0011001; //          E
    4'hF : prod_disp = 7'b0010010; //          F
    default : prod_disp = 7'b1111111;
endcase
endfunction

```

```

always @(posedge clk_1ms)
begin
    led_check = ~led_check;
    case (anode)
        4'b0111: anode <= 4'b1011;
        4'b1011: anode <= 4'b1101;
        4'b1101: anode <= 4'b1110;
        4'b1110: anode <= 4'b0111;
        default: anode <= 4'b0111;
    endcase
end

```

```

always @ (posedge clk_1ms)
begin
case (ps)
Reset:
begin
if (anode == 4'b1110)//one anode previously
dout <= 7'b0000011;//b
else if (anode == 4'b0111)
dout <= 7'b1000001;//u
else if (anode == 4'b1011)
dout <= 7'b0010001;//y
else dout <= 7'b1111111;//off
end
ProdSel:
begin
if (anode == 4'b1110)//one anode previously
begin
dout <= prod_disp(CurrProd, CurrPrice, anode);
end
if (anode == 4'b0111)
begin
dout <= 7'b1111111;
end
if (anode == 4'b1011)
begin
if (CurrPrice < 4'hA)
dout <= 7'b1000000; // 0
else if (CurrPrice == 4'bzzzz)//Hide for startup
dout <= 7'b1111111;
else dout <= 7'b1111001; // 1

end
if (anode == 4'b1101)
begin

```

```

        dout <= prod_disp(CurrProd, CurrPrice, anode);
    end
end
AmtSel:
begin
    if (anode == 4'b1110)//one anode previously
    begin
        dout <= prod_disp(CurrProd, CurrPrice, anode);
    end
    if (anode == 4'b0111)
    begin
        dout <= 7'b0111111;
    end
    if (anode == 4'b1011)
    begin
        if ((CurrPrice-money) < 4'hA) //Variable MSB of value
            dout <= 7'b1000000; // 0
        else dout <= 7'b1111001; // 1
        end
    if (anode == 4'b1101)
    begin
        dout <= prod_disp(CurrProd, (CurrPrice-money), anode); //Variable value
    end
end
end
Disp:
begin
    if (anode == 4'b1110)//one anode previously
        dout <= 7'b0100001;//d
    else if (anode == 4'b0111)
        dout <= 7'b1111001;//i
    else if (anode == 4'b1011)
        dout <= 7'b0010010;//s
    else dout <= 7'b0001100;//off
end
end

```

```

Chng:
begin
    if (anode == 4'b1110)//one anode previously
        dout <= 7'b1000110;//C
    else if (anode == 4'b0111)
        dout <= 7'b0001001;//H
    else if (anode == 4'b1011)//change doubles as refund also
        if ((money-CurrPrice) < 4'hA) //Variable MSB of value
            dout <= 7'b1000000; // 0
        else dout <= 7'b1111001; // 1
    else dout <= prod_disp(CurrProd, (money-CurrPrice), anode); //display remaining
change
end
Refund:
begin
    if (anode == 4'b1110)//one anode previously
        dout <= 7'b0101111;//r
    else if (anode == 4'b0111)
        dout <= 7'b0001110;//F
    else if (anode == 4'b1011)//change doubles as refund also
        if ((money) < 4'hA) //Variable MSB of value
            dout <= 7'b1000000; // 0
        else dout <= 7'b1111001; // 1
    else dout <= prod_disp(CurrProd, (money), anode); //display remaining change
end
Stock:
begin
    if (anode == 4'b1110)//one anode previously
        dout <= 7'b0101011;//n
    else if (anode == 4'b0111)
        dout <= 7'b0100011;//o
    else if (anode == 4'b1011)
        dout <= 7'b1111111;//off
    else //Product

```

```

begin
    case (CurrProd)
        4'hA : dout <= 7'b0001000; // A
        4'hB : dout <= 7'b0000011; // B
        4'hC : dout <= 7'b1000110; // C
        4'hD : dout <= 7'b0100001; // D
        4'hE : dout <= 7'b0000110; // E
        default: dout <= 7'b1111111;
    endcase
end
end
default:
begin
    dout <= 7'b1111111;
end
endcase
end
endmodule

```

7. RTL:

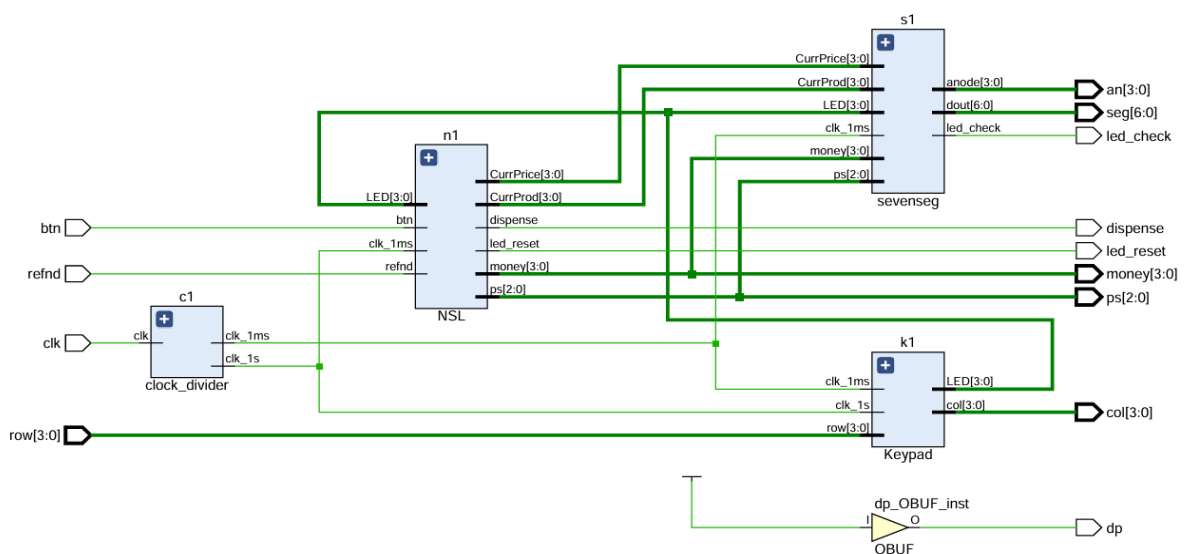


Fig. 3 RTL Diagram

8. Resource Utilization summary table:

Summary

Resource	Utilization	Available	Utilization %
LUT	215	20800	1.03
FF	119	41600	0.29
IO	37	106	34.91

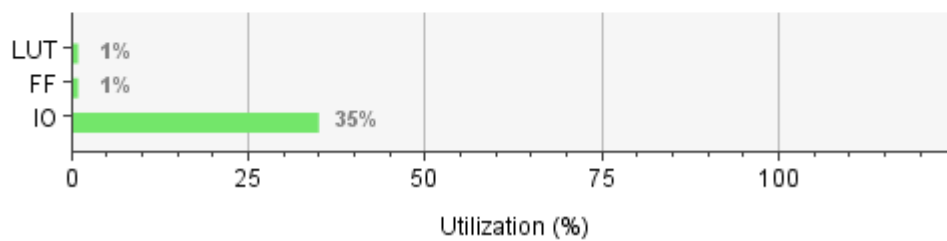


Fig. 4 Utilization Report

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 27.162 W (Junction temp exceeded!)

Design Power Budget: Not Specified

Power Budget Margin: N/A

Junction Temperature: 125.0°C

Thermal Margin: -75.8°C (-14.8 W)

Effective θ_{JA} : 5.0°C/W

Power supplied to off-chip devices: 0 W

Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

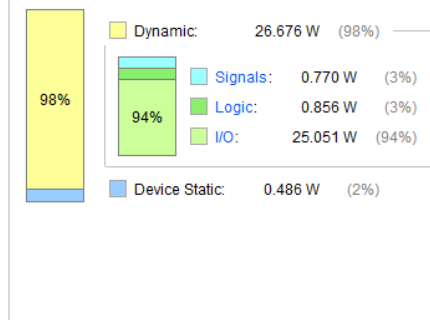


Fig. 5 Power Report

9. Output Waveform

Here, the price of the selected product C matches the money inserted, so the product is dispensed.

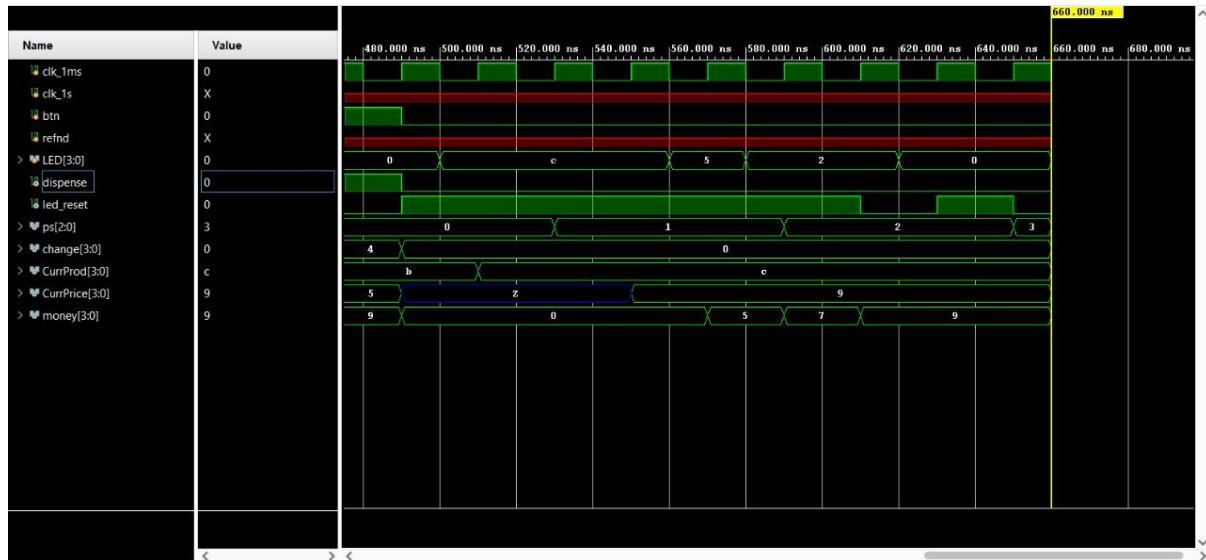


Fig. 6 Simulation of Product Dispense State (011)

Here, Product A is selected with a price of \$6. The customer inserts \$9, so the cost is covered and \$3 is returned as change.

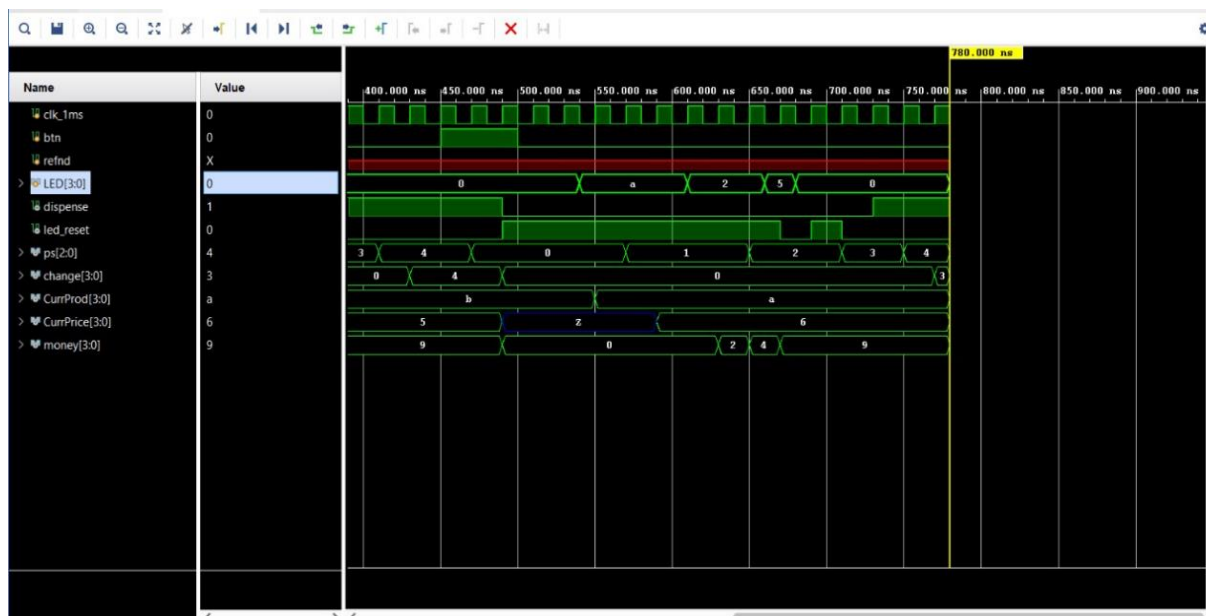


Fig. 7 Simulation of Product Change State (100)

Here, Product D is selected and \$2 is inserted. Since the refund button is pressed, the \$2 is returned as change and Product D is not dispensed.

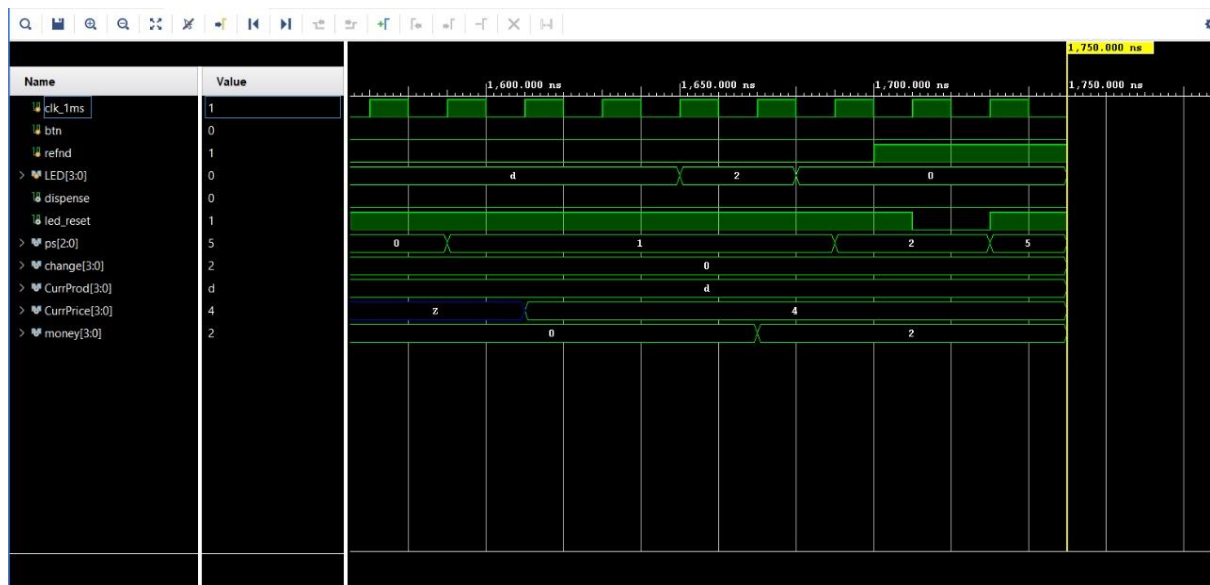


Fig. 8 Simulation of Product Refund State (101)

Here, Product E is Out of Stock therefore directly it goes to out of stock mode and does not accept money.

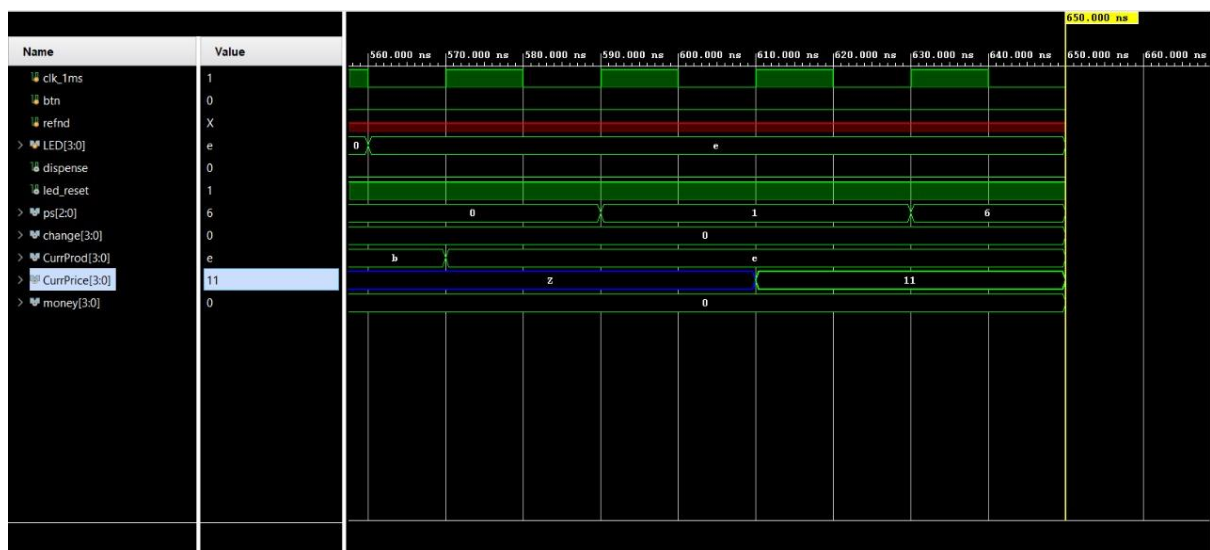


Fig. 9 Simulation of Out-of-Stock State (110)

10. Hardware Images:

- LD0, LD1, LD2- Shows Vending Machine's Present State
- LD4, LD5, LD6, LD7- Shows Amount Inserted

- LD13, LD14- to ensure readiness of the Vending Machine
- LD15- Glows when Product Dispensed

Reset State(000):

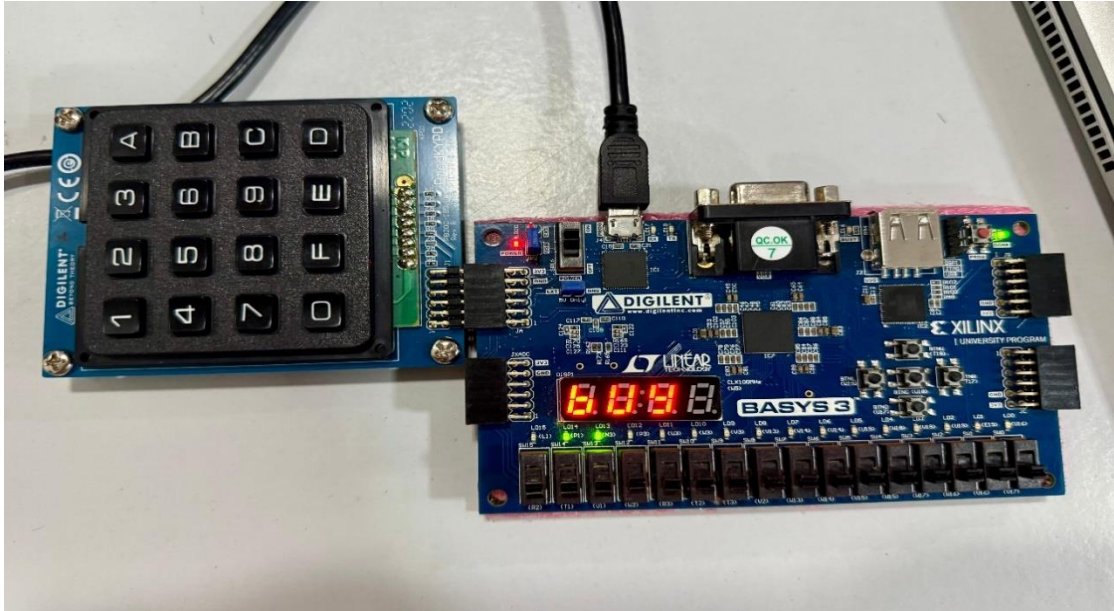


Fig. 10 Ready to Buy

Product Select(001):

Here, we have 5 products. Now user selected Product A, and its price is \$6.

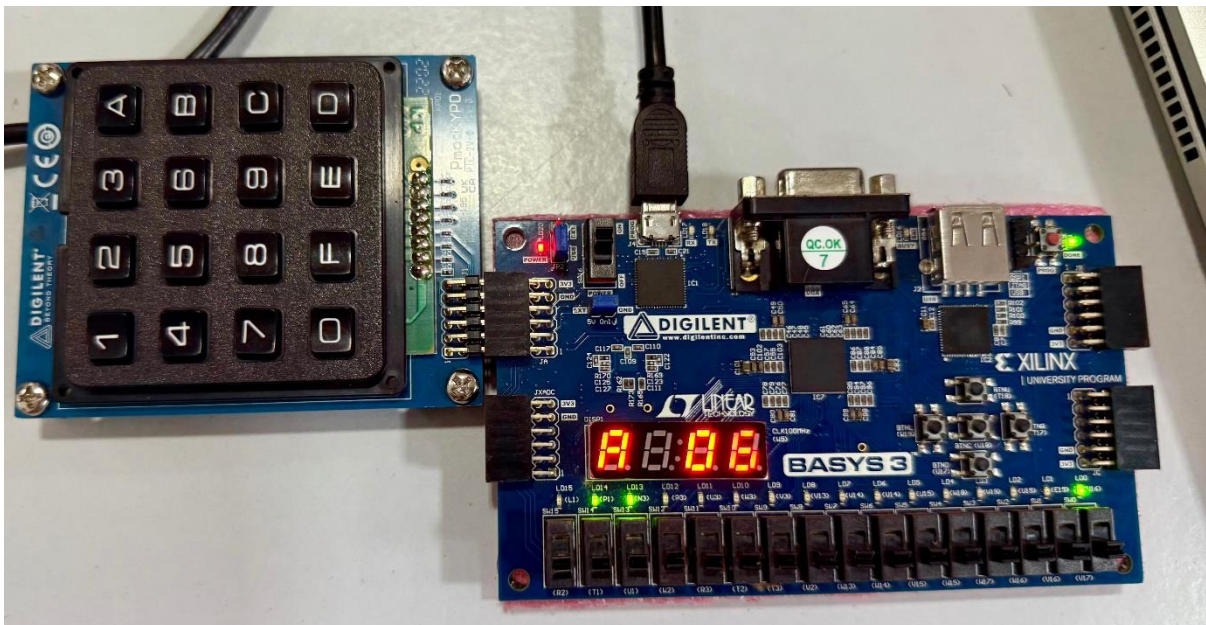


Fig. 11 Product Selection Mode(A Selected)

Amount Select (010):

Here, we inserted \$5 and remaining \$1 must be inserted to dispense item since price of product A is \$6. Also See LD4, LD6 glows meaning \$5(101) inserted

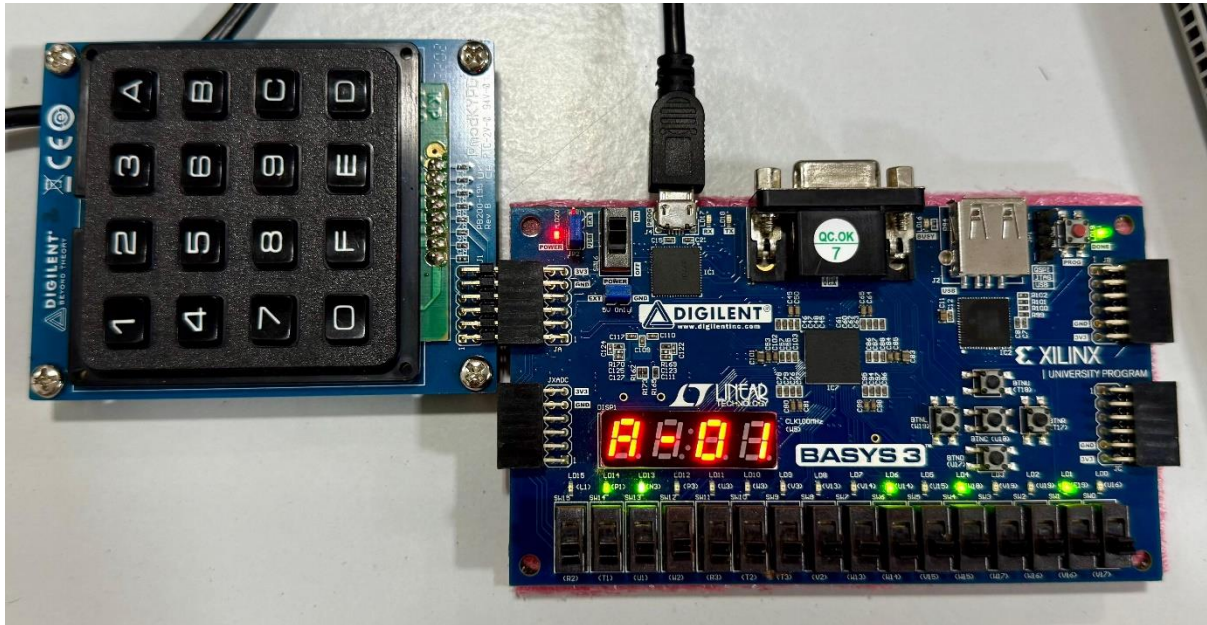


Fig. 12 Amount Selection Mode(\$5 Inserted)

Dispense State(011):

Therefore \$1 inserted and money is equal to cost, and the product A is dispensed. LD15 glows and display shows product dispensed.

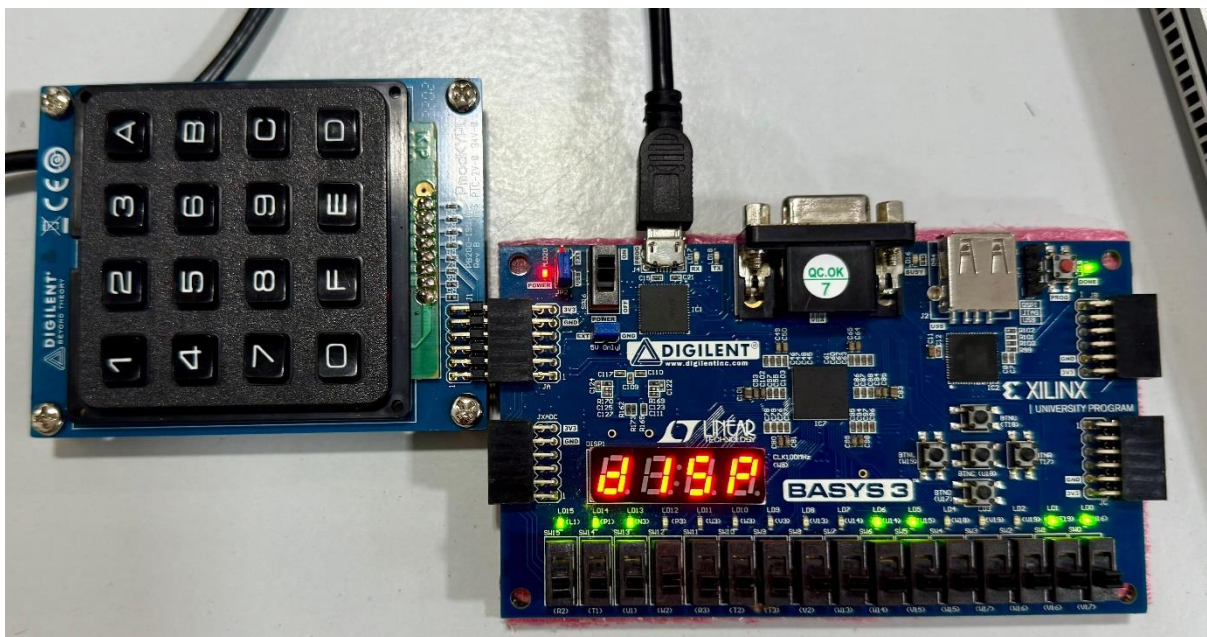


Fig. 13 Dispense Mode(Product A Dispensed)

Change State(100):

Now, the user has selected Product D, and its price is \$4.

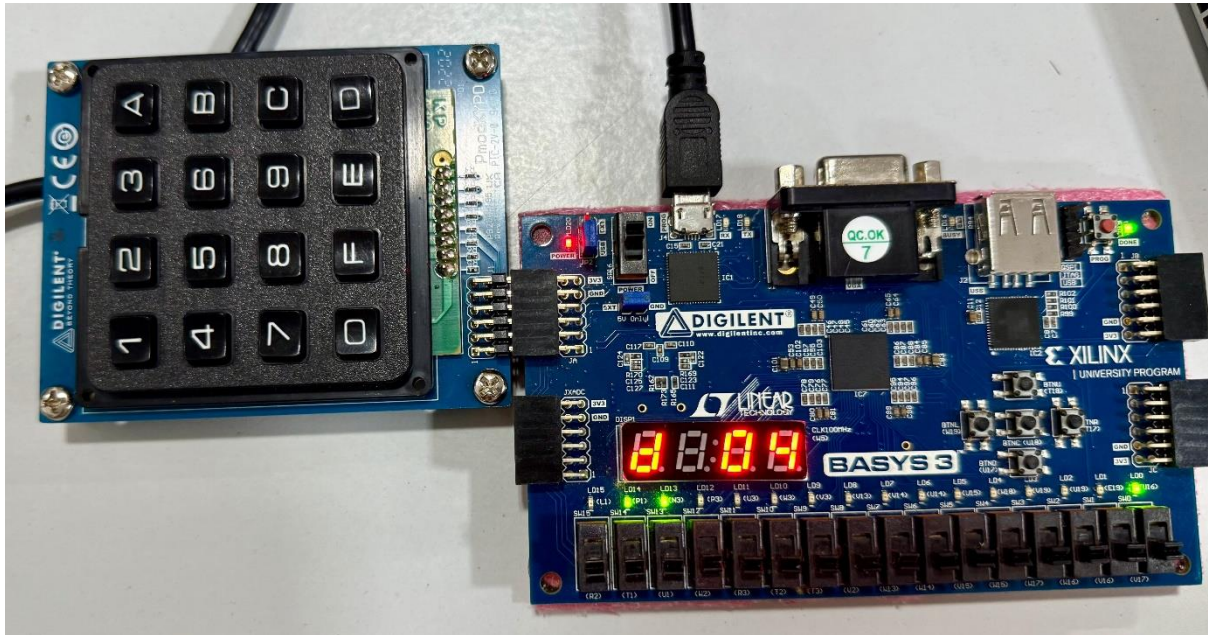


Fig. 14 Product Select Mode(Product D Selected)

Next, the user inserts \$5, the machine dispenses Product D, and \$1 is returned as change

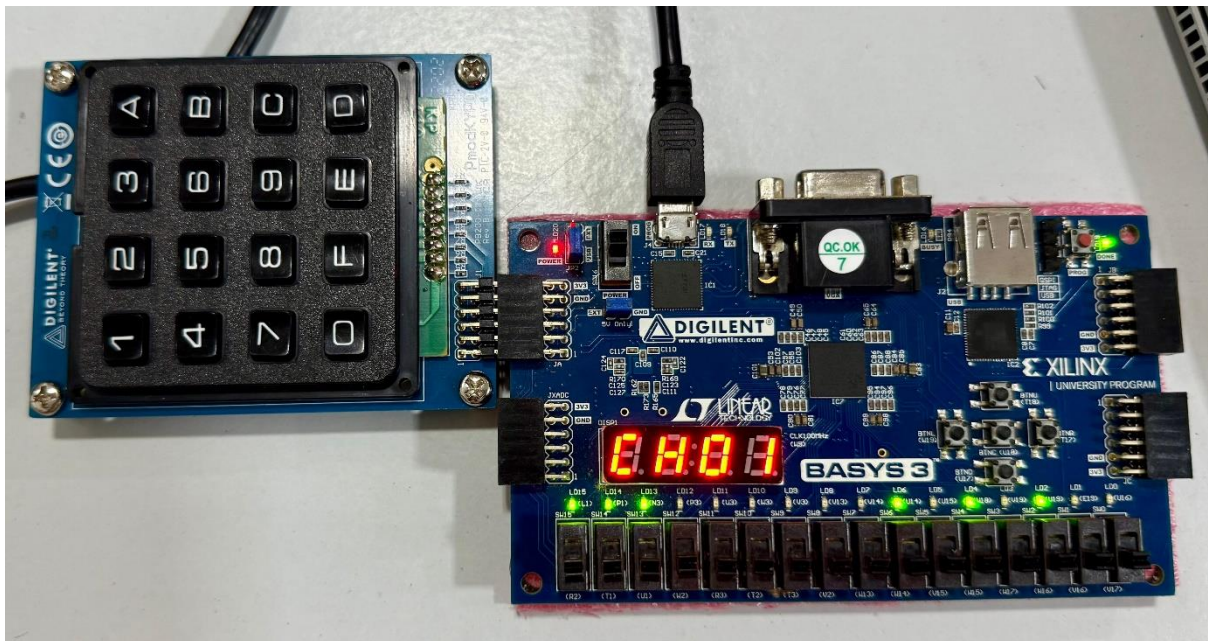


Fig. 15 Change Mode(\$1 Change returned)

Refund Mode(101):

The user selects Product B and inserts \$2. When the cancel button is pressed, Product B is not dispensed and the amount of \$2 is refunded.

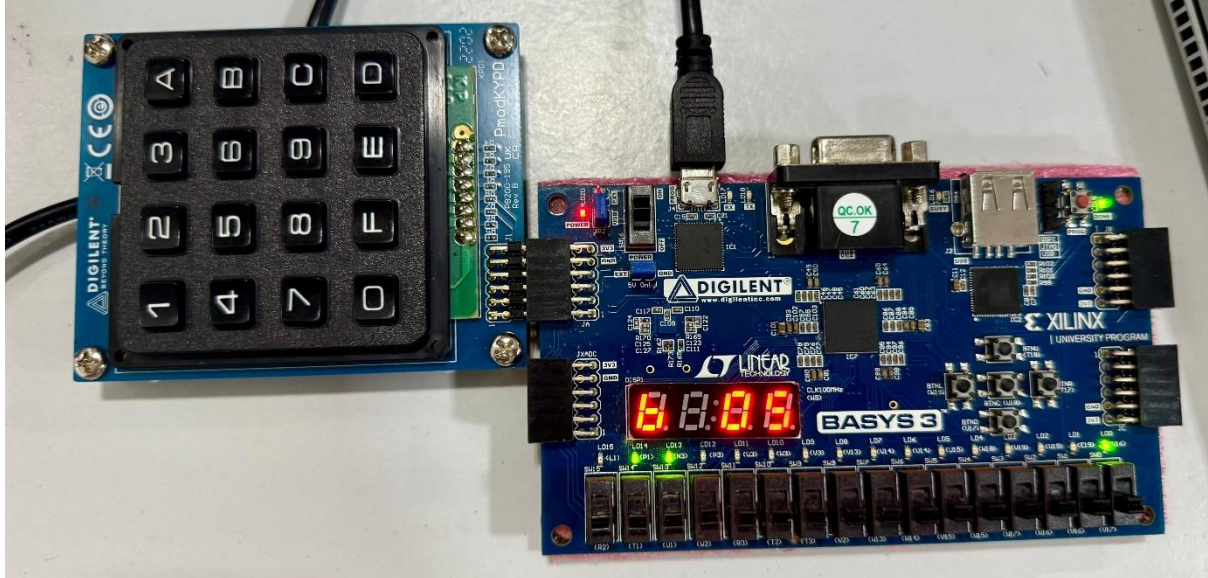


Fig. 16 Product B Selected

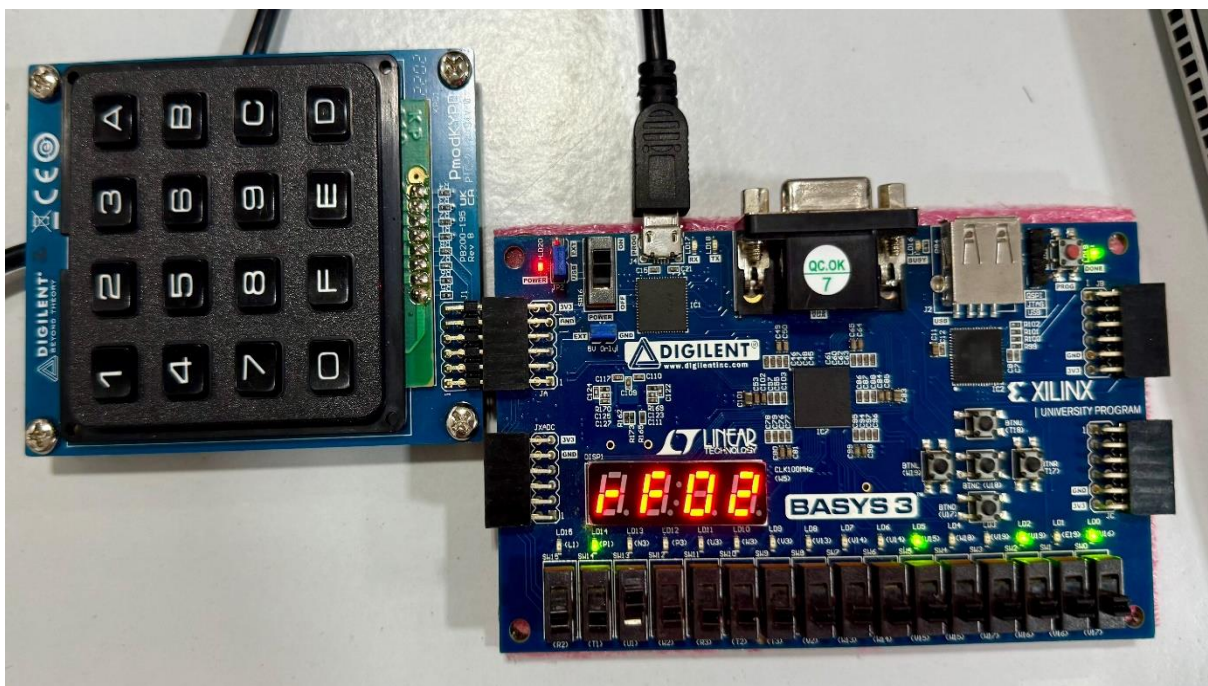


Fig. 17 Inserted \$2 is refunded and Product B not Dispensed

No Stock State(110):

The user selects Product E, but since it is out of stock, the machine displays 'Out of Stock'.

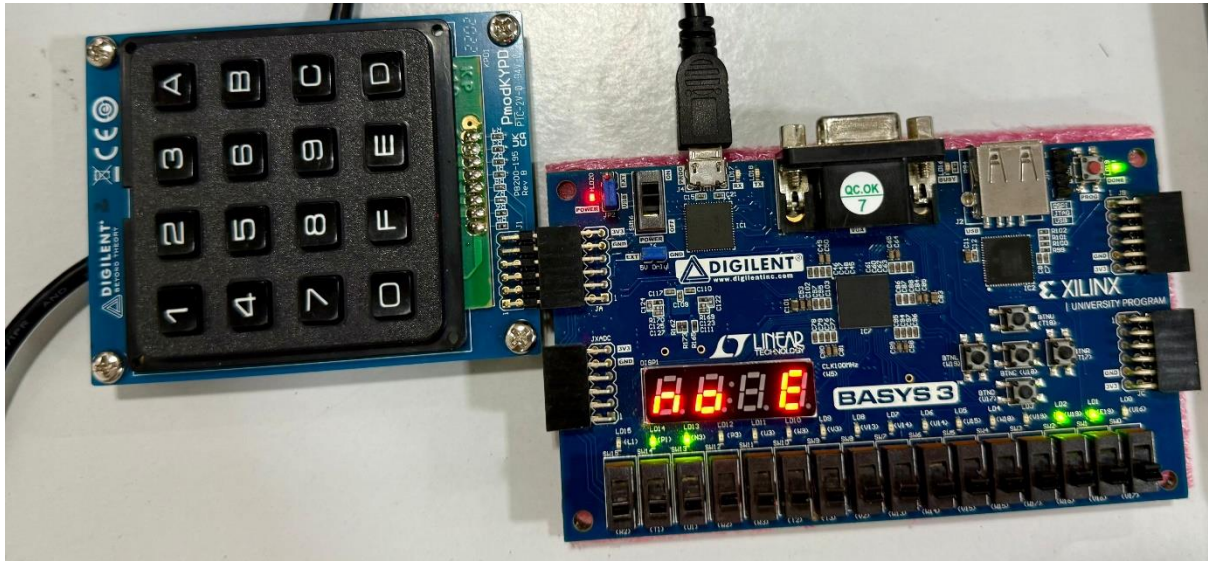


Fig. 18 No Stock E is Shown